

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по дисциплине

Вычислительная математика

Лабораторная работа №5

«Методы численного решения систем линейных
уравнений»

Вариант №15

Выполнил:

студент группы 5130201/40003

Гумницкий К. С.

Принял:

В. Г. Пак

«_____» _____ 2026 г.

Санкт-Петербург, 2026

1 Задание

1. Исследовать линейную систему методом Гаусса. Найти решение в случае его существования.
2. Решить приближённо линейную систему методами простой итерации, Якоби, Зейделя с данными начальными приближениями. Вычислить заданное число итераций или найти приближённое решение с данной точностью.

2 Цель работы

Научиться использовать методы численного решения систем линейных уравнений, научиться применять методы Гаусса, простой итерации, Якоби и Зейделя.

3 Ход работы

	0,97	0,05	-0,22	0,33	0,43
15	-0,22	0,45	0,08	-0,07	-1,8
	-0,33	-0,13	1,08	0,05	-0,8
	-0,08	-0,17	-0,29	0,67	1,7

Рис. 1: Вариант.

Для выполнения работы была написана программа на языке программирования Python. Исследование системы методом Гаусса происходит в функции `gauss(A, B)`, где `A` и `b` - соответствующие матрицы. Функция возвращает решение системы, если оно существует, или `None`, если оно отсутствует.

В функции применены следующие методы библиотеки `numpy` :

1. `np.array` - для перевода в массивы `numpy` .
2. `np.hstack` - объединяет вектор `A` и вектор `b` в одну матрицу.
3. `np.argmax` - ищет индекс максимального элемента.
4. `np.linalg.matrix_rank` - ищет ранг матрицы.

Функция `jacobi(A, b)` , где `A` и `b` - соответствующие матрицы, решает систему методом Якоби. Функция возвращает решение системы. В функции применены следующие методы библиотеки `numpy` :

1. `np.zeros` - возвращает новый массив указанной формы, заполненный нулями.
2. `np.diag` - извлечение диагонали из двумерной матрицы.
3. `np.dot` - вычисляет произведение матриц.
4. `np.linalg.norm` - вычисляет норму матрицы матриц.

Функция `seidel(A, b)` , где `A` и `b` - соответствующие матрицы, решает систему методом Зейделя. Функция возвращает решение системы. В функции применены те же методы библиотеки `numpy` , что и в функции `jacobi(A, b)` :

4 Результаты

Все три функции дали одинаковое решение системы.

$$\begin{pmatrix} -0.0180 \\ -3.6212 \\ -1.2323 \\ 1.0830 \end{pmatrix}$$

5 Приложение

```
1 import numpy as np
2
3 def gauss(A, b):
4     n = A.shape[0]
5
6     Ab = np.hstack([A, b.reshape(-1, 1)])
7
8     for i in range(n):
9         max_row = i + np.argmax(np.abs(Ab[i:, i]))
10        if np.abs(Ab[max_row, i]) < 1e-12:
11            continue
12
13        if max_row != i:
14            Ab[[i, max_row]] = Ab[[max_row, i]]
15            print(f" {i+1} {max_row+1}")
16
17        Ab[i] = Ab[i] / Ab[i, i]
18
19        for j in range(n):
20            if j != i:
21                Ab[j] = Ab[j] - Ab[j, i] * Ab[i]
22
23        rank_A = np.linalg.matrix_rank(A, tol=1e-10)
24        rank_Ab = np.linalg.matrix_rank(Ab, tol=1e-10)
25
26        if rank_A == rank_Ab:
27            if rank_A == n:
28                print(" .")
29                return Ab[:, -1]
30        else:
```

```

31         print(" .")
32         print(" :")
33         return Ab[:, -1]
34     else:
35         print(" .")
36         return None
37
38 def jacobi(A, b):
39     A = np.array(A, dtype=float)
40     b = np.array(b, dtype=float)
41     n = len(b)
42
43     x = np.zeros(n)
44
45     diag = np.diag(A)
46     inv_diag = 1 / diag
47
48     A_off = A - np.diag(diag)
49
50     for k in range(1000):
51         x_new = inv_diag * (b - np.dot(A_off, x))
52
53         if np.linalg.norm(x_new - x, np.inf) < 1e-6:
54             return x_new
55
56         x = x_new
57
58     return x
59
60 def seidel(A, b):
61     A = np.array(A, dtype=float)
62     b = np.array(b, dtype=float)

```

```

63     n = len(b)
64
65     x = np.zeros(n)
66
67     for k in range(1000):
68         x_old = x.copy()
69
70         for i in range(n):
71             x[i] = (b[i] - np.dot(A[i, :i], x[:i]) - np.dot(A[i, i+1:], x_old[i+1:])) / A[i, i]
72
73         if np.linalg.norm(x - x_old, np.inf) < 1e-6:
74             return x
75
76     return x
77
78
79 A = np.array([[0.97, 0.05, -0.22, 0.33], [-0.22, 0.45, 0.08, -0.07], [-0.33, -0.13, 1.08, 0.05], [-0.08,
      -0.17, -0.29, 0.67]])
80 b = np.array([0.43, -1.8, -0.8, 1.7])
81
82 print("")
83 x = gauss(A, b)
84 if x is not None:
85     print(" x:", x)
86
87 print()
88
89 print("")
90 x = jacobi(A, b)
91 print(":", x)
92
93 print()

```

```
94  
95 print("")  
96 x = seidel(A, b)  
97 print(":", x)
```

Листинг 1: Файл Lab5.py